



Enterprise-Grade AI Engineering Portfolio

Product Requirements Document (PRD) & Technical Requirements Document (TRD)

Version 2.0 — Exhaustive Edition

Document Classification: Internal — Portfolio Engineering

Prepared By: Senior Product Manager & Lead Software Architect

Target Audience: Solo Developer / Owner

Deployment Target: Vercel (Free Tier) + Firebase Spark Plan (Free Tier)

Last Updated: 2026

TABLE OF CONTENTS

1. [Executive Summary](#)
2. [Product Vision & Goals](#)
3. [User Personas](#)
4. [Epics & User Stories](#)
5. [Feature Specifications \(Public-Facing\)](#)
6. [Feature Specifications \(Admin CMS\)](#)
7. [Non-Functional Requirements](#)
8. [Success Metrics & KPIs](#)
9. [Roadmap & Phasing](#)
10. [TRD — Technology Stack](#)
11. [TRD — System Architecture](#)
12. [TRD — Database Schema](#)
13. [TRD — Firebase Security Rules](#)
14. [TRD — Admin Authentication & Security](#)
15. [TRD — API Design & Server Actions](#)
16. [TRD — CI/CD Pipeline Strategy](#)
17. [TRD — SEO & Performance Strategy](#)
18. [TRD — File & Asset Management](#)
19. [TRD — Error Handling & Edge Cases](#)
20. [TRD — Phase 2: AI Chatbot Integration](#)

21. [Appendix A — Project File Structure](#)

22. [Appendix B — Environment Variables](#)

PART I — PRODUCT REQUIREMENTS DOCUMENT (PRD)

1. Executive Summary

This document specifies the complete product and technical requirements for a **"High-Signal" Software Engineering Portfolio Web Application**. This is not a standard static portfolio — it is a living, self-managed, content-rich technical demonstration engineered to:

- **Bypass ATS systems entirely** by providing a direct, memorable URL to Engineering Managers
- **Demonstrate frontend mastery** through flawless performance, responsiveness, and animations
- **Demonstrate backend architecture** through a fully functional Firebase-backed CMS
- **Demonstrate AI engineering depth** through a Phase 2 Agentic RAG chatbot
- **Generate qualified leads** through contact form routing and engagement analytics
- **Rank on search engines** through complete technical SEO coverage

The entire application must be **100% free to build and host**, leveraging Vercel's Hobby plan and Firebase's Spark (free) plan. Every feature on the admin panel must be fully editable by the owner without touching source code or triggering redeployments.

2. Product Vision & Goals

2.1 Vision Statement

"To serve as an always-on, self-updating technical partner that convinces elite Engineering Managers of the candidate's capabilities within the first 90 seconds of a site visit — without a single conversation needing to happen."

2.2 Strategic Goals

Goal	Rationale
Score 100/100 on Google Lighthouse (all four categories)	Proves performance engineering competency immediately

Goal	Rationale
Maintain zero-downtime on free infrastructure	Proves cost-optimization and infrastructure judgment
Enable full content management with zero redeployments	Proves backend architecture competency
Generate >40% CV download rate	Primary conversion event for lead capture
Be indexable and rankable for AI Engineer + location keywords	Long-term inbound pipeline for opportunities

3. User Personas

Persona 1 — The Engineering Manager (Primary)

Attribute	Detail
Name	"Alex"
Role	Engineering Manager / Tech Lead at a funded startup or FAANG-adjacent company
Location	USA, EU, or UAE (USD-pegged remote role)
Time on Site	60-120 seconds maximum before deciding to keep exploring or leave
Tech Savvy	High — will open DevTools, check Lighthouse, and inspect the source
Pain Points	Wading through identical templated portfolios; no verifiable impact; broken links; poor mobile UX
Goal	Quickly assess actual coding ability, system design judgment, and real-world delivery
What Impresses Them	Clean architecture, live GitHub stats, video walkthroughs, fast load times, no layout shift

Persona 2 — The Technical Recruiter (Secondary)

Attribute	Detail
Name	"Sam"
Role	External or internal technical recruiter sourcing for senior AI roles
Time on Site	30-60 seconds
Tech Savvy	Medium — can read skill tags and project names but won't dive into code
Goal	Extract key skills, contact the candidate, and download the CV
What Impresses Them	Clear skill tags, prominent CV download, easy-to-use contact form, professional design

Persona 3 — The Portfolio Owner / Admin (Tertiary)

Attribute	Detail
Name	The Candidate
Role	AI Engineer maintaining this portfolio as a living document
Frequency of Use	Weekly content updates; occasional project additions
Goal	Add new projects, publish blog posts, moderate comments, and track site analytics — all from <code>/admin</code> — without touching code
Pain Points	Hardcoded data requiring redeployments; no analytics visibility; no moderation tools

4. Epics & User Stories

Epic 1: First Impression & Navigation

E1-S1 — Hero Section:

As a visitor, I want to see a high-impact hero section with the candidate's name, title, a one-line value proposition, a dynamic SVG skills matrix, a "Download CV" button, and social links — so that I can immediately understand who this person is and what they offer.

Acceptance Criteria:

- Hero loads above the fold on all screen sizes
- "Download CV" triggers a PDF download from Firebase Storage
- SVG skills matrix animates on first load
- All social links open in `_blank` with `rel="noopener noreferrer"`

E1-S2 — Navigation:

As a visitor, I want a sticky, minimal top navigation bar with smooth scroll links to all major sections and a visible Dark/Light mode toggle, so that I can navigate the site without confusion.

Acceptance Criteria:

- Navbar is `position: sticky; top: 0` with backdrop blur
- Active section is highlighted using IntersectionObserver
- Dark/Light toggle persists across sessions via localStorage
- Mobile nav collapses into a hamburger menu

Epic 2: Credibility & Experience

E2-S1 — Vertical Timeline:

As an Engineering Manager, I want to scroll through a visually engaging chronological timeline of the candidate's academic and work history, so that I can verify career progression and identify relevant experience quickly.

Acceptance Criteria:

- Timeline renders vertically with alternating left/right layout on desktop; single column on mobile
- Each entry includes: organization name, date range, role/degree title, and 3-5 bullet-point descriptions
- Events are tagged as "Academic" or "Corporate" with distinct visual treatments
- Timeline data is fetched from Firestore and fully editable from the Admin CMS
- Animate entries into view on scroll using IntersectionObserver

E2-S2 — Skills Matrix:

As a technical evaluator, I want to see the candidate's skills organized by category (e.g., Languages, AI/ML, Cloud, Frameworks), with proficiency indicators, so that I can assess stack fit in seconds.

Acceptance Criteria:

- Skills grouped into at minimum 5 categories

- Each skill displays a proficiency level (e.g., Expert / Proficient / Familiar)
- SVG or CSS-based visual (NO Canvas) — animates on load
- Skills data is editable from Admin CMS

Epic 3: Tiered Project Showcase

E3-S1 — Tier 1 Flagship Projects:

As a technical evaluator, I want to see the top 3 flagship AI projects in an expanded, premium card format with embedded Loom videos, live GitHub stats, and demo links, so that I can deeply assess the candidate's highest-impact work.

Acceptance Criteria:

- Three projects: (1) Agentic RAG Workflow, (2) SLM Fine-tuning Pipeline, (3) Deep Learning Deforestation Model
- Each card contains:
 - Project title, 3-sentence description, and full tech stack tags
 - Embedded Loom iframe (responsive `aspect-ratio: 16/9`) for Demo + Architecture walkthrough
 - Live GitHub stats: star count and commit count fetched from GitHub REST API
 - "Live Demo" and "View Code" CTA buttons
- GitHub stats are fetched server-side (Next.js Route Handler) to hide the API token
- Stats refresh via ISR every 1 hour
- If GitHub API fails, display last cached values (graceful degradation)

E3-S2 — Tier 2 Production Apps:

As a visitor, I want to browse 4 production application cards in a clean grid, each showing a brief description, tech stack, and repo link, so that I can see the breadth of the candidate's shipping experience.

Acceptance Criteria:

- 2-column grid on desktop, 1-column on mobile
- Each card: project name, 1-sentence description, array of tech stack pill badges, GitHub link, optional live demo link
- Cards pull from Firestore `projects` collection filtered by `tier: 2`
- Hovering a card reveals a subtle animated overlay

E3-S3 — Tier 3 Utilities:

As a visitor, I want to see a compact, scannable list of minor tools and utilities the candidate has built, so that I can gauge productivity and side-project culture.

Acceptance Criteria:

- Simple list or compact 3-column grid
- Each entry: name, one-line description, link (GitHub or live)
- Filtered from Firestore `projects` collection by `tier: 3`

Epic 4: Blog Engine

E4-S1 — Blog Feed:

As a technical visitor, I want to browse a feed of the candidate's technical articles, so that I can assess their depth of thinking and communication ability.

Acceptance Criteria:

- Paginated grid of blog post cards (title, date, estimated read time, 1-sentence excerpt)
- Only `published: true` posts are visible to public
- Cards link to individual post pages at `/blog/[slug]`
- Feed is SSG at build time with ISR (revalidates every 5 minutes for new posts)

E4-S2 — Individual Blog Post:

As a reader, I want to read a full Markdown-rendered blog post with syntax-highlighted code blocks and proper typography, so that I can absorb the technical content.

Acceptance Criteria:

- Markdown rendered using `react-markdown` + `remark-gfm` + `rehype-highlight`
- Code blocks have syntax highlighting and a "Copy" button
- Estimated read time displayed in the header (words / 200 wpm)
- Table of contents auto-generated from headings
- Social share buttons (Twitter/X, LinkedIn, copy link)
- Structured data (`application/ld+json`) for `Article` schema on every post

E4-S3 — Public Commenting:

As a reader, I want to leave a comment on a blog post without creating an account, so that I can engage with the author and other readers.

Acceptance Criteria:

- Comment form: name field (required) + comment textarea (required)
- On submit: comment written to Firestore with `status: "pending"`
- User sees a success toast: "Comment submitted for review"
- Only `status: "approved"` comments are displayed publicly
- Comment list displays: name, date, and comment content

- Comments load below the post content without blocking page render (client-side fetch)

Epic 5: Contact & Lead Generation

E5-S1 — Contact Form:

As a recruiter, I want to send a direct message to the candidate via a simple contact form, without logging in or leaving the site, so that I can initiate a conversation with minimal friction.

Acceptance Criteria:

- Form fields: Name (required), Email (required, validated), Company (optional), Message (required, min 20 chars)
- Client-side validation with inline error states before submission
- On submit: triggers EmailJS or Resend API server action to deliver email to candidate's inbox
- Submission stored in Firestore `contact_submissions` collection for admin review
- Rate-limited to prevent spam: max 3 submissions per IP per hour (via Next.js middleware)
- On success: form replaced with a personalized confirmation message
- On failure: error message with retry option; no data loss

Epic 6: Admin CMS — Authentication

E6-S1 — Hidden Admin Route:

As the Admin, I want to navigate to `/admin` which is not linked anywhere on the public site, and be presented with a secure login screen, so that the admin panel is discoverable only to me.

Acceptance Criteria:

- `/admin` is not in the sitemap, not in `robots.txt` as accessible, and has no inbound links from public pages
- Unauthenticated visitors to `/admin/**` are immediately redirected to `/admin/login`
- Login page has no site navigation, no footer — minimal design
- Login page uses meta `noindex, nofollow`

E6-S2 — Secure Login:

As the Admin, I want to authenticate using my credentials, with multi-layer security including rate limiting and anomaly detection, so that no one else can access the CMS.

Acceptance Criteria (detailed in TRD Section 14):

- Firebase Auth Email/Password with a hardcoded single admin UID
- Login attempts rate-limited: after 5 failed attempts, IP blocked for 15 minutes
- Brute-force lockout stored in Firestore

- Session managed via Firebase ID token stored in an HttpOnly cookie
- Session auto-expires after 1 hour of inactivity; requires re-authentication
- All admin API routes verify the Firebase ID token server-side before executing
- Login events (success and failure) are logged to Firestore `admin_audit_log` collection

Epic 7: Admin CMS — Content Management

E7-S1 — Admin Dashboard:

As the Admin, I want a clean dashboard showing site stats (total projects, total posts, pending comments, pending contact messages) so that I have an at-a-glance overview upon login.

E7-S2 — Project CRUD:

As the Admin, I want to create, read, update, and delete projects across all three tiers from a form interface, so that the project showcase updates instantly without redeployment.

Acceptance Criteria:

- Table listing all projects with columns: Title, Tier, CreatedAt, Actions (Edit | Delete)
- "New Project" button opens a form modal or dedicated page
- Form fields: title, tier (1/2/3), description, tech stack (tag input), GitHub repo URL, live demo URL, Loom video ID (Tier 1 only), image upload (Firebase Storage), published toggle
- On save: Firestore document updated; public pages revalidate via Next.js `revalidatePath` / `revalidateTag`
- Delete prompts a confirmation modal before executing
- Optimistic UI: list updates immediately while write completes

E7-S3 — Blog CRUD:

As the Admin, I want to write, edit, preview, and publish blog posts using a rich Markdown editor, so that I can maintain a technical blog without any external CMS.

Acceptance Criteria:

- Blog list table with columns: Title, Slug, Status (Draft/Published), CreatedAt, Actions
- Editor uses `@uiw/react-md-editor` or `tiptap` for Markdown with live preview
- Auto-generates SEO slug from title (user can override)
- Fields: title, slug, markdown content, excerpt, cover image (Firebase Storage upload), tags, published toggle
- Saving as draft does not publish publicly; published toggle controls visibility
- Image uploads in the editor insert the Firebase Storage URL inline

E7-S4 — Comment Moderation:

As the Admin, I want to see all pending comments across all blog posts and approve or delete

them with one click, so that I can maintain a spam-free comments section.

Acceptance Criteria:

- Moderation queue shows all `{status: "pending"}` comments, sorted by date (oldest first)
- Each row: author name, comment excerpt, post title, date, Approve button, Delete button
- Approve: updates `{status}` to `"approved"` in Firestore — comment becomes immediately public
- Delete: removes document from Firestore — no recovery
- Counts on dashboard badge update in real time using Firestore `onSnapshot`

E7-S5 — Timeline Management:

As the Admin, I want to add, edit, reorder, and delete timeline events, so that my experience section always reflects my current history.

E7-S6 — Contact Submissions Inbox:

As the Admin, I want to view all submitted contact form messages in the admin panel, mark them as read, and delete them, so that I never miss a lead even if the email fails.

E7-S7 — CV/Resume Upload:

As the Admin, I want to upload a new version of my CV PDF directly from the admin panel and have the "Download CV" button on the public site immediately serve the new file, so that I never have to touch Firebase Storage manually.

Acceptance Criteria:

- File picker for PDF upload
 - Validates file type (PDF only) and size (max 5MB)
 - Uploads to Firebase Storage at a fixed path (`/cv/resume.pdf`) — overwrites previous
 - Public "Download CV" button always resolves to this fixed Storage URL
-

5. Feature Specifications (Public-Facing)

5.1 Open Graph & Social Sharing

Every public page must have complete OG and Twitter Card meta tags:

```
html
```

```

<!-- Homepage -->
<meta property="og:title" content="[Name] | AI Engineer" />
<meta property="og:description" content="[One-line value prop]" />
<meta property="og:image" content="https://[domain]/og-image.png" />
<meta property="og:url" content="https://[domain]" />
<meta name="twitter:card" content="summary_large_image" />

<!-- Blog post pages – dynamically generated -->
<meta property="og:title" content="[Post Title] | [Name]" />
<meta property="og:type" content="article" />
<meta property="og:image" content="[Cover Image URL or generated OG image]" />

```

OG images for blog posts are generated dynamically using Next.js `ImageResponse` from `next/og`. Each generated image includes the post title and the candidate's name/avatar on a branded background.

5.2 Dark/Light Mode

- System preference detected on first visit using `prefers-color-scheme` media query
- User override stored in `localStorage` under key `theme`
- Theme applied as a `data-theme` attribute on the `<html>` element
- CSS variables (custom properties) drive all colors — no hardcoded color values anywhere in components
- Toggle animation: smooth 200ms transition on all color properties
- No FOUC (Flash of Unstyled Content): theme is read and applied in a `<script>` tag before React hydrates (in `app/layout.tsx`)

5.3 Custom 404 Page

- Fully custom `/not-found.tsx` page
- Interactive element: e.g., a small terminal animation that types out "404: page not found" with a blinking cursor
- A navigation prompt to return home or browse projects
- Matches site theme (dark/light responsive)
- Does NOT load heavy assets — must be lightweight

5.4 Accessibility (A11y) Requirements

- All images have meaningful `alt` attributes
- Color contrast ratio $\geq 4.5:1$ for normal text (WCAG AA)
- All interactive elements are keyboard-navigable

- Modals trap focus correctly
 - ARIA labels on icon-only buttons
 - `prefers-reduced-motion` media query respected: all animations disabled when user preference is set
 - Skip-to-content link as the first focusable element on every page
-

6. Feature Specifications (Admin CMS)

6.1 Admin Panel Layout

The admin panel at `/admin` is a separate layout (`app/admin/layout.tsx`) with:

- Its own navigation sidebar (Projects, Blog, Comments, Timeline, Contact Inbox, CV Upload, Settings)
- No public site header/footer
- Session status indicator (logged in as: `[email]`, logout button)
- Responsive: sidebar collapses on mobile

6.2 Real-Time Updates

Where applicable, admin views use Firestore `onSnapshot` listeners for real-time updates:

- Comment moderation queue updates live as new comments arrive
- Dashboard stats badge counts update live
- Contact submissions inbox updates live

6.3 Admin Settings Page

A dedicated Settings page accessible from the sidebar allows the Admin to update:

- Display name and profile photo (uploaded to Firebase Storage)
- Social links (GitHub, LinkedIn, Twitter, email)
- Hero section tagline and sub-headline
- Contact section email override
- Site-wide SEO title suffix

All settings stored in a single Firestore document: `settings/global`.

7. Non-Functional Requirements

7.1 Performance

Metric	Target
Lighthouse Performance Score	100
Lighthouse Accessibility Score	100
Lighthouse Best Practices Score	100
Lighthouse SEO Score	100
First Contentful Paint (FCP)	< 1.2s
Largest Contentful Paint (LCP)	< 2.5s
Cumulative Layout Shift (CLS)	< 0.1
Time to Interactive (TTI)	< 3.5s
Total Blocking Time (TBT)	< 200ms

7.2 Responsiveness

All components must be designed mobile-first:

- Breakpoints: `sm: 640px`, `md: 768px`, `lg: 1024px`, `xl: 1280px`
- No horizontal scrolling at any breakpoint
- Touch targets minimum 44x44px
- Font sizes minimum 16px on mobile (no zoom-in required to read)
- Images use `next/image` with proper `sizes` attribute to serve responsive images

7.3 Security

- All admin routes protected by server-side session verification
- Firebase Security Rules enforce all access patterns (see Section 13)
- API tokens never exposed to the client
- Environment variables never leaked to the browser (prefix rule enforced via ESLint plugin)
- CSP (Content Security Policy) headers configured in `next.config.ts`
- Rate limiting on contact form and admin login routes via Next.js Middleware
- HTTPS enforced (Vercel provides automatically)

7.4 Free Tier Compliance

Service	Free Tier Limit	Expected Usage
Vercel Hobby	100GB bandwidth/month	Safely within for portfolio traffic
Firebase Auth	10,000 authentications/month	1 admin user — irrelevant
Firestore Reads	50,000/day	Negligible for portfolio; CDN caching reduces reads
Firestore Writes	20,000/day	Admin only — negligible
Firebase Storage	1GB stored, 10GB/month transfer	CV (~200KB) + images — safe
GitHub API (unauthenticated)	60 req/hour	Handled by server-side ISR caching
EmailJS Free	200 emails/month	Sufficient for contact form

8. Success Metrics & KPIs

8.1 Technical KPIs (Measured via Vercel Analytics & Lighthouse CI)

KPI	Target	Measurement Method
Lighthouse Performance	100/100	Lighthouse CI in GitHub Actions
Lighthouse Accessibility	100/100	Lighthouse CI in GitHub Actions
Lighthouse SEO	100/100	Lighthouse CI in GitHub Actions
Lighthouse Best Practices	100/100	Lighthouse CI in GitHub Actions
FCP	< 1.2s	Vercel Web Vitals
CLS	< 0.1	Vercel Web Vitals

8.2 Engagement KPIs (Measured via Vercel Analytics)

KPI	Target	Notes
CV Download CTR	> 40% of visitors	Primary conversion event
Loom Video Play Rate	> 60% on Tier 1 cards	Indicates deep engagement
Average Session Duration	> 90 seconds	Validates content quality
Blog Post Reads	> 30% of visitors navigate to blog	Content marketing goal
Contact Form Submissions	100% successfully delivered	Zero failed sends
Bounce Rate	< 50%	Quality traffic assumption

9. Roadmap & Phasing

Phase 1 — Core Portfolio (Current)

Feature	Priority	Estimated Effort
Project scaffold (Next.js + Firebase + Tailwind)	P0	4 hours
Hero section + SVG skills matrix	P0	6 hours
Timeline component	P0	4 hours
Tier 1/2/3 Project showcase	P0	8 hours
Blog engine (feed + post + Markdown)	P0	8 hours
Public commenting	P1	4 hours
Contact form (EmailJS)	P0	3 hours
Admin auth + protected routes	P0	6 hours
Admin CRUD (Projects, Blog)	P0	10 hours
Admin moderation + inbox	P1	4 hours
Dark/Light mode	P1	2 hours

Feature	Priority	Estimated Effort
Custom 404 page	P2	2 hours
SEO + OG tags + structured data	P0	4 hours
Performance optimization (Lighthouse 100)	P0	6 hours
CI/CD pipeline (Lighthouse CI)	P1	3 hours

Total Estimated Phase 1: ~74 hours

Phase 2 — AI Integration

Feature	Priority	Estimated Effort
Agentic RAG Chatbot (CV + READMEs + blogs)	P0	20 hours
Chatbot UI widget (bottom-right floating button)	P0	4 hours
Document ingestion pipeline	P0	8 hours
Admin chatbot knowledge base management	P1	4 hours

Total Estimated Phase 2: ~36 hours

PART II — TECHNICAL REQUIREMENTS DOCUMENT (TRD)

10. TRD — Technology Stack

10.1 Full Stack Rationale

Layer	Technology	Version	Rationale
Frontend Framework	Next.js (App Router)	15.x	SSG/SSR/ISR, server components, server actions, built-in optimization
UI Language	React	19.x	Component model, hydration, concurrent features

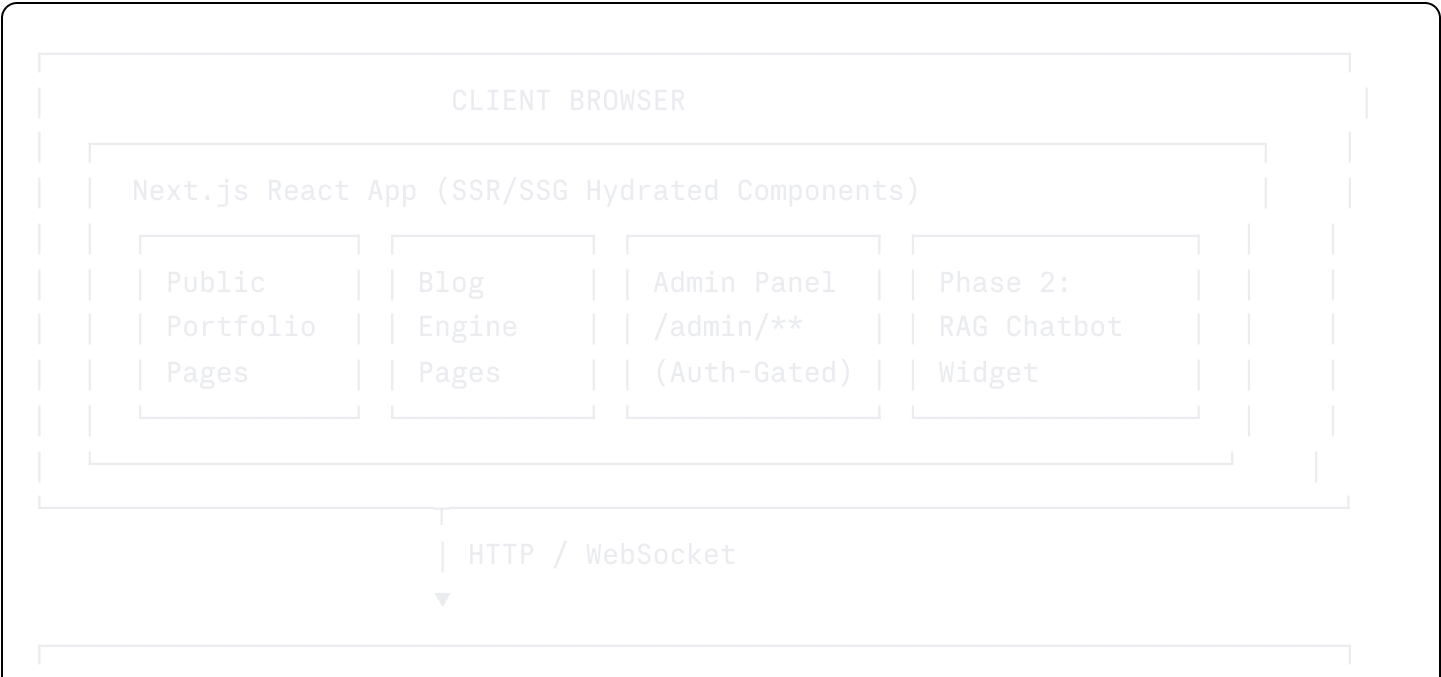
Layer	Technology	Version	Rationale
Styling	TailwindCSS	3.x	Utility-first, purges unused CSS, excellent DX
Language	TypeScript	5.x	Type safety, better DX, fewer runtime errors
Database	Firebase Firestore	v10 (modular)	NoSQL, real-time, generous free tier, no server needed
Auth	Firebase Auth	v10	Simple email/password, JWT tokens, easy session management
Storage	Firebase Storage	v10	File uploads, PDF hosting, image hosting
Email	EmailJS	v4	Client-configurable, free tier (200/mo), no backend required
Markdown	react-markdown + remark-gfm + rehype-highlight	Latest	Standards-compliant Markdown rendering with syntax highlighting
Rich Text Editor	@uiw/react-md-editor	Latest	Markdown editor with live preview for Admin
Analytics	Vercel Analytics	Built-in	Zero config, privacy-friendly, no cookie banner
Hosting	Vercel (Hobby)	—	Free, edge CDN, automatic HTTPS, preview deployments
CI/CD	GitHub Actions + Vercel	—	Native integration, automatic previews
Code Quality	ESLint + Prettier + Husky	Latest	Pre-commit hooks, consistent formatting
Icons	Lucide React	Latest	Lightweight, tree-shakeable, accessible
Animations	Framer Motion	Latest	Production-grade animations, reduced-motion support
Forms	React Hook Form + Zod	Latest	Performant forms with schema validation

10.2 Rendering Strategy Per Route

Route	Strategy	Reason
<code>/</code> (Homepage)	SSG + ISR (1 hour)	Static with dynamic GitHub stats
<code>/blog</code> (Feed)	SSG + ISR (5 min)	Updates when new posts published
<code>/blog/[slug]</code>	SSG (generateStaticParams) + ISR (5 min)	SEO critical — static + fast updates
<code>/admin/**</code>	Dynamic (no caching)	Auth-protected, always fresh
<code>/api/github-stats</code>	Route Handler + ISR (1 hour)	Server-side GitHub token, cached
<code>/api/contact</code>	Route Handler, no caching	Live form submission
<code>/sitemap.xml</code>	Dynamic Route Handler	Auto-generated from Firestore blog slugs
<code>/robots.txt</code>	Static	No-index admin, index everything else

11. TRD — System Architecture

11.1 High-Level Architecture Diagram





11.2 Data Flow — Public Page Load (Homepage)

1. Browser requests "/"
2. Vercel CDN serves pre-rendered HTML (SSG hit — sub-50ms)
3. React hydrates in browser
4. Client-side: Firestore onSnapshot NOT used on public pages (SSG data)

5. GitHub stats: fetched server-side via Route Handler, cached for 1 hour (ISR)
6. Blog posts: fetched at build time via generateStaticParams, cached

11.3 Data Flow — Admin CRUD Operation

1. Admin navigates to `/admin/projects/new`
2. Middleware checks `HttpOnly` cookie → extracts Firebase ID token
3. Next.js Server Component verifies token via Firebase Admin SDK
4. If invalid → redirect to `/admin/login`
5. If valid → render admin form
6. Admin submits form → Server Action invoked
7. Server Action: re-verifies token → writes to Firestore
8. Server Action: calls `revalidatePath('/')` to bust ISR cache
9. Admin sees optimistic UI update immediately
10. Next build incremental regeneration triggered for affected static pages

11.4 Data Flow — Contact Form Submission

1. User fills out contact form → client-side Zod validation
2. On valid submit → Server Action called
3. Server Action: checks rate limit (Firestore `'rate_limits'` collection)
4. If rate limit exceeded → return error to client
5. If within limit: write submission to Firestore `'contact_submissions'`
6. Server Action: call EmailJS API (or Resend) with submission data
7. EmailJS delivers email to candidate inbox
8. Server Action: update rate limit counter in Firestore
9. Return success → client shows confirmation message

12. TRD — Database Schema

12.1 Collection: `projects`

typescript

```

interface Project {
  id: string; // Auto-generated Firestore ID
  title: string; // "Agentic RAG Workflow Engine"
  slug: string; // "agentic-rag-workflow-engine"
  tier: 1 | 2 | 3; // Controls display section
  shortDescription: string; // 1-sentence teaser (for Tier 2/3 cards)
  fullDescription: string; // 3-sentence detailed description (for Tier 1)
  techStack: string[]; // ["Python", "LangChain", "Pinecone", "FastAPI"]
  githubRepo: string; // "username/repo-name" (used for API stats)
  liveDemoUrl: string | null; // "https://demo.example.com" or null
  loomVideoId: string | null; // "abc123xyz" – Tier 1 only, null for others
  coverImageUrl: string | null; // Firebase Storage URL for project thumbnail
  featured: boolean; // Pin to top within tier
  published: boolean; // Draft/live toggle
  order: number; // Manual sort order within tier
  createdAt: Timestamp;
  updatedAt: Timestamp;
}

```

Indexes Required:

- Composite: `(tier ASC, order ASC)` (for tiered listings)
- Single: `(published)` (filter published only)

12.2 Collection: `blogs`

typescript

```

interface Blog {
  id: string; // Auto-generated
  title: string; // "Building Agentic RAG Pipelines with LangChain"
  slug: string; // "building-agentic-rag-pipelines-langchain" (unique)
  excerpt: string; // 1-2 sentence summary for feed card
  markdownContent: string; // Full Markdown body
  coverImageUrl: string | null; // Firebase Storage URL
  tags: string[]; // ["AI", "RAG", "LangChain", "Python"]
  readTimeMinutes: number; // Auto-calculated: Math.ceil(wordCount / 200)
  published: boolean; // Draft vs live
  publishedAt: Timestamp | null; // Set when published:true first set
  createdAt: Timestamp;
  updatedAt: Timestamp;
}

```

Sub-collection: `blogs/{blogId}/comments`

typescript

```
interface Comment {
  id: string; // Auto-generated
  authorName: string; // Public display name
  content: string; // Comment body (max 1000 chars)
  status: "pending" | "approved" | "rejected";
  ipHash: string; // SHA-256 of submitter IP (for abuse tracking, not
  createdAt: Timestamp;
}
```

Indexes Required:

- Single: `published` + `publishedAt DESC` (for public feed, sorted by date)
- Sub-collection: `comments.status` + `comments.createdAt` (for moderation queue)

12.3 Collection: `timeline_events`

typescript

```
interface TimelineEvent {
  id: string;
  type: "academic" | "corporate" | "achievement";
  title: string; // "BSc Computer Science" | "AI Engineer Intern"
  organization: string; // "University of X" | "FFC SAP"
  location: string; // "Lahore, PK" | "Remote"
  dateRange: string; // "Sep 2021 - Jun 2025"
  startDate: Timestamp; // For chronological sorting
  endDate: Timestamp | null; // null = current
  description: string[]; // Array of bullet point strings
  logoUrl: string | null; // Organization logo from Firebase Storage
  order: number; // Manual override for sort order
}
```

Index: `startDate DESC` (chronological order)

12.4 Collection: `contact_submissions`

typescript

```
interface ContactSubmission {
  id: string;
  name: string;
  email: string;
  company: string | null;
  message: string;
  status: "unread" | "read" | "archived";
  emailDelivered: boolean; // Whether EmailJS/Resend succeeded
  ipHash: string; // SHA-256 of submitter IP
  userAgent: string; // Browser user agent string
  createdAt: Timestamp;
}
```

12.5 Collection: **settings**

Single document: **settings/global**

typescript

```
interface GlobalSettings {
  displayName: string; // "Muhammad [Name]"
  tagline: string; // "AI Engineer | Builder of Intelligent Systems"
  subHeadline: string; // "I build production-grade AI systems..."
  avatarUrl: string; // Firebase Storage URL
  cvFileUrl: string; // Firebase Storage URL for /cv/resume.pdf
  email: string; // Contact routing email
  github: string; // GitHub profile URL
  linkedin: string; // LinkedIn profile URL
  twitter: string | null; // Twitter/X profile URL
  seoTitleSuffix: string; // "| AI Engineer & ML Practitioner"
  metaDescription: string; // Global site meta description
  updatedAt: Timestamp;
}
```

12.6 Collection: **admin_audit_log**

typescript

```
interface AuditLogEntry {
  id: string;
  event: "login_success" | "login_failure" | "logout" | "project_created"
        | "project_updated" | "project_deleted" | "blog_published"
        | "comment_approved" | "comment_deleted" | "cv_uploaded";
  adminUid: string | null; // null on login_failure (no authenticated UID)
  ipHash: string;
  userAgent: string;
  metadata: Record<string, unknown>; // Additional context (e.g., projectId, blogSlug)
  createdAt: Timestamp;
}
```

12.7 Collection: `rate_limits`

typescript

```
interface RateLimitEntry {
  id: string; // SHA-256 of IP + endpoint (e.g., "contact" or "login")
  count: number; // Number of requests in current window
  windowStart: Timestamp; // Start of current 1-hour window
  blockedUntil: Timestamp | null; // For login: block timestamp after 5 failures
}
```

13. TRD — Firebase Security Rules

javascript

```

// firestore.rules
// COMPLETE PRODUCTION RULES
// Replace YOUR_ADMIN_UID with the Firebase Auth UID of your admin account.
// To find your UID: Firebase Console → Authentication → Users → copy UID column.

rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {

    // _____
    // HELPER FUNCTIONS
    // _____

    // True only if the requesting user is the master admin
    function isAdmin() {
      return request.auth != null
        && request.auth.uid == 'YOUR_ADMIN_UID'
        && request.auth.token.email_verified == true;
    }

    // True for any authenticated request (currently only admin exists)
    function isAuthenticated() {
      return request.auth != null;
    }

    // Validates that a comment creation request is safe
    function isValidCommentCreate() {
      let data = request.resource.data;
      return data.status == 'pending'
        && data.content is string
        && data.content.size() > 0
        && data.content.size() <= 1000
        && data.authorName is string
        && data.authorName.size() > 0
        && data.authorName.size() <= 100
        && data.keys().hasOnly(['status', 'content', 'authorName', 'ipHash', 'crea

    }

    // Validates that a contact submission is safe
    function isValidContactSubmission() {
      let data = request.resource.data;
      return data.status == 'unread'
        && data.name is string

```

```

    && data.email is string
    && data.message is string
    && data.message.size() >= 20
    && data.message.size() <= 2000;
}

// -----
// PROJECTS
// -----
match /projects/{projectId} {
  // Public: read only published projects
  allow read: if resource.data.published == true || isAdmin();
  // Admin: full write access
  allow create, update, delete: if isAdmin();
}

// -----
// TIMELINE EVENTS
// -----
match /timeline_events/{eventId} {
  allow read: if true;
  allow create, update, delete: if isAdmin();
}

// -----
// BLOGS
// -----
match /blogs/{blogId} {
  // Public: read only published posts
  allow read: if resource.data.published == true || isAdmin();
  allow create, update, delete: if isAdmin();

  // COMMENTS (sub-collection)
  match /comments/{commentId} {
    // Public: read only approved comments
    allow read: if resource.data.status == 'approved' || isAdmin();

    // Public: create comment (strictly validated, always pending)
    allow create: if isValidCommentCreate();

    // Only admin can approve or delete
    allow update, delete: if isAdmin();
  }
}

```

```
// _____
// CONTACT SUBMISSIONS
// _____
match /contact_submissions/{submissionId} {
  // Public: can create (submit contact form)
  allow create: if isValidContactSubmission();
  // Only admin can read, update (mark read), or delete
  allow read, update, delete: if isAdmin();
}

// _____
// SETTINGS (global config)
// _____
match /settings/{docId} {
  // Public can read global settings (needed for site rendering)
  allow read: if true;
  // Only admin can write settings
  allow write: if isAdmin();
}

// _____
// AUDIT LOG (admin write, no public access)
// _____
match /admin_audit_log/{logId} {
  allow read, write: if isAdmin();
}

// _____
// RATE LIMITS (server-side only via Admin SDK)
// _____
match /rate_limits/{limitId} {
  // No direct client access – all reads/writes via Admin SDK in Server Actions
  allow read, write: if false;
}

// _____
// CATCH-ALL: Deny everything not explicitly matched
// _____
match /{document=**} {
  allow read, write: if false;
}
```

```
}  
}
```

Firebase Storage Rules:

```
javascript
```

```
// storage.rules
rules_version = '2';
service firebase.storage {
  match /b/{bucket}/o {

    // CV/Resume: Public read, admin write only
    match /cv/{fileName} {
      allow read: if true;
      allow write: if request.auth != null
                    && request.auth.uid == 'YOUR_ADMIN_UID'
                    && request.resource.contentType == 'application/pdf'
                    && request.resource.size < 5 * 1024 * 1024; // 5MB limit
    }

    // Images: Public read, admin write only
    match /images/{allPaths=**} {
      allow read: if true;
      allow write: if request.auth != null
                   && request.auth.uid == 'YOUR_ADMIN_UID'
                   && request.resource.contentType.matches('image/.*')
                   && request.resource.size < 10 * 1024 * 1024; // 10MB limit
    }

    // Avatars: Public read, admin write only
    match /avatars/{fileName} {
      allow read: if true;
      allow write: if request.auth != null
                   && request.auth.uid == 'YOUR_ADMIN_UID'
                   && request.resource.contentType.matches('image/.*')
                   && request.resource.size < 2 * 1024 * 1024; // 2MB limit
    }

    // Deny all other paths
    match /{allPaths=**} {
      allow read, write: if false;
    }
  }
}
```

14. TRD — Admin Authentication & Security

14.1 Authentication Architecture

The admin authentication system uses multiple layers of defense:

```
Layer 1: Route-level — Next.js Middleware redirects unauthenticated users
Layer 2: Component-level — Server Components verify token before rendering
Layer 3: Action-level — Every Server Action re-verifies the token independently
Layer 4: Database-level — Firebase Security Rules enforce UID match
Layer 5: Audit-level — All auth events logged to Firestore audit_log
```

14.2 Login Flow (Step-by-Step)

```
Step 1: User navigates to /admin/login
Step 2: Middleware checks for session cookie → not found → allow /login to render
Step 3: User enters email + password → client-side validation
Step 4: Check Firestore rate_limits for this IP hash:
    - If blockedUntil > now() → return "Account temporarily locked" error
    - If count >= 5 in current window → set blockedUntil = now() + 15min
Step 5: Call Firebase Auth signInWithEmailAndPassword()
Step 6: On Firebase auth failure → increment rate_limit count → log to audit_log
(event: login_failure) → return generic error "Invalid credentials"
Step 7: On Firebase auth success → verify UID === YOUR_ADMIN_UID
    - If UID mismatch → sign out → return "Access denied" → log to audit_log
Step 8: Call Firebase Auth getIdToken() to retrieve ID token (JWT)
Step 9: POST ID token to Server Action /api/auth/session
Step 10: Server Action: verify token via Firebase Admin SDK (verifyIdToken())
Step 11: Server Action: create HttpOnly, Secure, SameSite=Strict cookie named
__session
    containing the ID token, expires in 1 hour
Step 12: Log to audit_log (event: login_success)
Step 13: Redirect to /admin/dashboard
```

14.3 Session Verification (Every Admin Request)

```
typescript
```

```
// lib/auth/verify-session.ts
import { adminAuth } from '@lib/firebase/admin';
import { cookies } from 'next/headers';
import { redirect } from 'next/navigation';

export async function verifyAdminSession() {
  const cookieStore = await cookies();
  const sessionCookie = cookieStore.get('__session')?.value;

  if (!sessionCookie) {
    redirect('/admin/login');
  }

  try {
    const decodedToken = await adminAuth.verifyIdToken(sessionCookie);

    // CRITICAL: verify UID matches admin
    if (decodedToken.uid !== process.env.ADMIN_UID) {
      redirect('/admin/login');
    }

    // CRITICAL: verify token is not older than 1 hour
    const tokenAge = Date.now() / 1000 - decodedToken.iat;
    if (tokenAge > 3600) {
      redirect('/admin/login');
    }

    return decodedToken;
  } catch (error) {
    redirect('/admin/login');
  }
}
```

14.4 Security Headers (next.config.ts)

typescript

```
const securityHeaders = [
  { key: 'X-DNS-Prefetch-Control', value: 'on' },
  { key: 'Strict-Transport-Security', value: 'max-age=63072000; includeSubDomains; p' },
  { key: 'X-Frame-Options', value: 'SAMEORIGIN' },
  { key: 'X-Content-Type-Options', value: 'nosniff' },
  { key: 'Referrer-Policy', value: 'strict-origin-when-cross-origin' },
  { key: 'Permissions-Policy', value: 'camera=(), microphone=(), geolocation=()' },
  {
    key: 'Content-Security-Policy',
    value: [
      "default-src 'self'",
      "script-src 'self' 'unsafe-eval' 'unsafe-inline' https://www.loom.com https://",
      "style-src 'self' 'unsafe-inline'",
      "img-src 'self' data: blob: https://firebasestorage.googleapis.com https://ava",
      "media-src 'self' https://www.loom.com",
      "frame-src https://www.loom.com",
      "connect-src 'self' https://firestore.googleapis.com https://firebase.googleap",
      "font-src 'self'",
    ].join('; ')
  }
];
```

14.5 Rate Limiting Middleware

typescript

```

// middleware.ts
import { NextResponse } from 'next/server';
import type { NextRequest } from 'next/server';

const PROTECTED_PATHS = ['/admin'];
const RATE_LIMITED_PATHS = ['/api/contact', '/admin/login'];

export async function middleware(request: NextRequest) {
  const { pathname } = request.nextUrl;

  // 1. Protect /admin routes (except /admin/login)
  if (pathname.startsWith('/admin') && pathname !== '/admin/login') {
    const session = request.cookies.get('__session');
    if (!session) {
      return NextResponse.redirect(new URL('/admin/login', request.url));
    }
    // Token verification happens at Server Component level (middleware can't use Ad
  }

  // 2. Add security headers to all responses
  const response = NextResponse.next();
  securityHeaders.forEach(({ key, value }) => {
    response.headers.set(key, value);
  });

  return response;
}

export const config = {
  matcher: ['/(?!_next/static|_next/image|favicon.ico|.*\\.png$).*.'],
};

```

15. TRD — API Design & Server Actions

15.1 Route Handlers

GET /api/github-stats?repo={owner/repo}

- Fetches star count and commit count from GitHub REST API
- Uses **GITHUB_TOKEN** env variable (server-side only)
- Response cached with **revalidate: 3600** (1 hour ISR)

- Returns: `{ stars: number, commits: number, lastUpdated: string }`
- On error: returns last cached value or `{ stars: null, commits: null, error: true }`

`GET /sitemap.xml`

- Dynamically generates XML sitemap
- Includes: all static pages + all published blog slugs from Firestore
- Excludes: `/admin/**` routes
- Sets `Cache-Control: public, max-age=86400` (revalidates daily)

`GET /robots.txt`

- Disallows: `/admin/`
- Allows: everything else
- Includes sitemap URL

15.2 Server Actions

`createProject(formData: ProjectFormData)`

- Verifies admin session
- Validates with Zod schema
- Writes to Firestore
- Calls `revalidatePath('/')` and `revalidatePath('/blog')`
- Returns: `{ success: true, projectId: string }` or `{ success: false, error: string }`

`submitContactForm(formData: ContactFormData)`

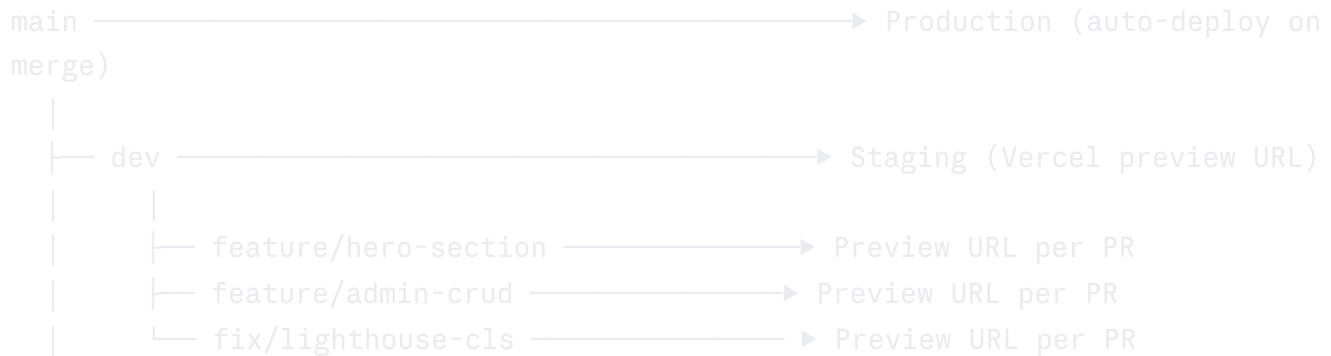
- Validates Zod schema
- Checks rate limit (3 per IP per hour)
- Writes to Firestore `contact_submissions`
- Calls EmailJS/Resend API
- Returns: `{ success: true }` or `{ success: false, error: string }`

`approveComment(blogId: string, commentId: string)`

- Verifies admin session
- Updates `status: "approved"` in Firestore
- Logs to `audit_log`
- Returns success/error

16. TRD — CI/CD Pipeline Strategy

16.1 Branching Strategy



Rules:

- Direct pushes to **main** are disabled (GitHub branch protection)
- All changes go through PR → **dev** → merge to **main**
- Linear commit history enforced (squash-merge only)

16.2 Pre-Commit Hooks (Husky + lint-staged)

```
json
// package.json - relevant section
{
  "scripts": {
    "lint": "next lint",
    "type-check": "tsc --noEmit",
    "format:check": "prettier --check .",
    "test": "vitest run"
  },
  "lint-staged": {
    "*.ts,tsx": ["eslint --fix", "prettier --write"],
    "*.json,md,css": ["prettier --write"]
  }
}
```

```
bash
```

```
# .husky/pre-commit
#!/usr/bin/env sh
. "$(dirname -- "$0")/_/husky.sh"

npx lint-staged
npm run type-check
```

16.3 GitHub Actions Workflows

Workflow 1: `ci.yml` — Runs on every PR

yaml

```

name: CI
on:
  pull_request:
    branches: [main, dev]

jobs:
  quality:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: '20'
          cache: 'npm'
      - run: npm ci
      - run: npm run type-check
      - run: npm run lint
      - run: npm run format:check

  lighthouse:
    runs-on: ubuntu-latest
    needs: quality
    steps:
      - uses: actions/checkout@v4
      - name: Wait for Vercel Preview
        uses: patrickedqvist/wait-for-vercel-preview@v1.3.1
        id: vercel-preview
        with:
          token: ${ secrets.GITHUB_TOKEN }
          max_timeout: 300

      - name: Run Lighthouse CI
        uses: treosh/lighthouse-ci-action@v11
        with:
          urls: ${ steps.vercel-preview.outputs.url }
          uploadArtifacts: true
          temporaryPublicStorage: true
          configPath: '.lighthouserc.json'

# Fails PR if Lighthouse score < 95 on any category

```

.lighthouserc.json

Workflow 2: `deploy.yml` — Production deploy on merge to main

yml

```
name: Deploy to Production
on:
  push:
    branches: [main]

jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Deploy to Vercel
        uses: amondnet/vercel-action@v25
        with:
          vercel-token: ${ secrets.VERCEL_TOKEN }
          vercel-org-id: ${ secrets.VERCEL_ORG_ID }
          vercel-project-id: ${ secrets.VERCEL_PROJECT_ID }
          vercel-args: '--prod'
```

16.4 Vercel Project Configuration (`vercel.json`)

json

```
{
  "framework": "nextjs",
  "buildCommand": "npm run build",
  "devCommand": "npm run dev",
  "installCommand": "npm ci",
  "headers": [
    {
      "source": "/(.*)",
      "headers": [
        { "key": "X-Content-Type-Options", "value": "nosniff" },
        { "key": "X-Frame-Options", "value": "SAMEORIGIN" }
      ]
    },
    {
      "source": "/admin/(.*)",
      "headers": [
        { "key": "X-Robots-Tag", "value": "noindex, nofollow" }
      ]
    }
  ]
}
```

17. TRD — SEO & Performance Strategy

17.1 Metadata Configuration (Next.js App Router)

typescript

```
// app/layout.tsx – Root metadata
export const metadata: Metadata = {
  metadataBase: new URL('https://yourdomain.com'),
  title: { default: '[Name] | AI Engineer', template: '%s | [Name]' },
  description: 'AI Engineer specializing in Agentic RAG, LLM Fine-tuning, and product development',
  keywords: ['AI Engineer', 'Machine Learning', 'RAG', 'LangChain', 'Remote AI Jobs'],
  authors: [{ name: '[Name]', url: 'https://yourdomain.com' }],
  creator: '[Name]',
  openGraph: {
    type: 'website',
    locale: 'en_US',
    url: 'https://yourdomain.com',
    siteName: '[Name] | AI Engineer Portfolio',
    images: [{ url: '/og-image.png', width: 1200, height: 630 }],
  },
  twitter: {
    card: 'summary_large_image',
    creator: '@yourtwitterhandle',
    images: ['/og-image.png'],
  },
  robots: {
    index: true,
    follow: true,
    googleBot: { index: true, follow: true, 'max-image-preview': 'large' },
  },
  verification: {
    google: 'your-google-search-console-verification-code',
  },
};
```

17.2 Structured Data (JSON-LD)

Homepage — Person schema:

json

```
{
  "@context": "https://schema.org",
  "@type": "Person",
  "name": "[Full Name]",
  "jobTitle": "AI Engineer",
  "url": "https://yourdomain.com",
  "sameAs": ["https://github.com/...", "https://linkedin.com/in/..."],
  "knowsAbout": ["Machine Learning", "Large Language Models", "RAG Systems"]
}
```

Blog Post — Article schema:

```
json
{
  "@context": "https://schema.org",
  "@type": "Article",
  "headline": "[Post Title]",
  "author": { "@type": "Person", "name": "[Name]" },
  "datePublished": "[ISO Date]",
  "dateModified": "[ISO Date]",
  "image": "[Cover Image URL]",
  "url": "https://yourdomain.com/blog/[slug]"
}
```

17.3 Image Optimization Strategy

- All images use `next/image` with explicit `width`, `height`, and `sizes`
- Images stored in Firebase Storage are served via CDN URLs
- Profile/avatar image: WebP format, 400x400px, pre-generated
- Blog cover images: WebP format, 1200x630px (OG image standard)
- `priority` prop on hero section image (above-the-fold)
- `loading="lazy"` implicit on all below-fold images

17.4 Font Strategy

- Fonts loaded via `next/font/google` (zero layout shift)
- Font subsetting enabled (only required Unicode ranges)
- `font-display: swap` configured automatically by next/font
- Maximum 2 font families: one display font, one body font

17.5 Bundle Optimization

typescript

```
// next.config.ts
const nextConfig: NextConfig = {
  experimental: {
    optimizePackageImports: ['lucide-react', 'framer-motion', '@firebase/firestore'],
  },
  images: {
    remotePatterns: [
      { protocol: 'https', hostname: 'firebasestorage.googleapis.com' },
      { protocol: 'https', hostname: 'avatars.githubusercontent.com' },
    ],
    formats: ['image/avif', 'image/webp'],
  },
  compress: true,
};
```

18. TRD — File & Asset Management

18.1 Firebase Storage Folder Structure

```

Firebase Storage Bucket
├─ cv/
│   └─ resume.pdf                ← Always at this fixed path
├─ images/
│   ├─ projects/
│   │   └─ {projectId}-cover.webp
│   │   └─ ...
│   └─ blog/
│       └─ {blogSlug}-cover.webp
│       └─ ...
└─ avatars/
    └─ profile.webp              ← Admin profile photo
```

18.2 CV Download Implementation

typescript

```
// The "Download CV" button resolves to a predictable Firebase Storage URL:
// https://firebasestorage.googleapis.com/v0/b/{BUCKET}/o/cv%2Fresume.pdf?alt=media

// Component:
<a
  href={settings.cvFileUrl}
  download="[Name]-CV.pdf"
  target="_blank"
  rel="noopener noreferrer"
  onClick={() => analytics.track('cv_download')}
>
  Download CV
</a>
```

19. TRD — Error Handling & Edge Cases

19.1 Error Boundary Strategy

- Global `error.tsx` in `app/` catches unexpected React errors
- Each major section has its own `error.tsx` so a failed section doesn't crash the entire page
- `loading.tsx` skeleton UIs for all data-fetched sections

19.2 GitHub API Failure Handling

```
typescript

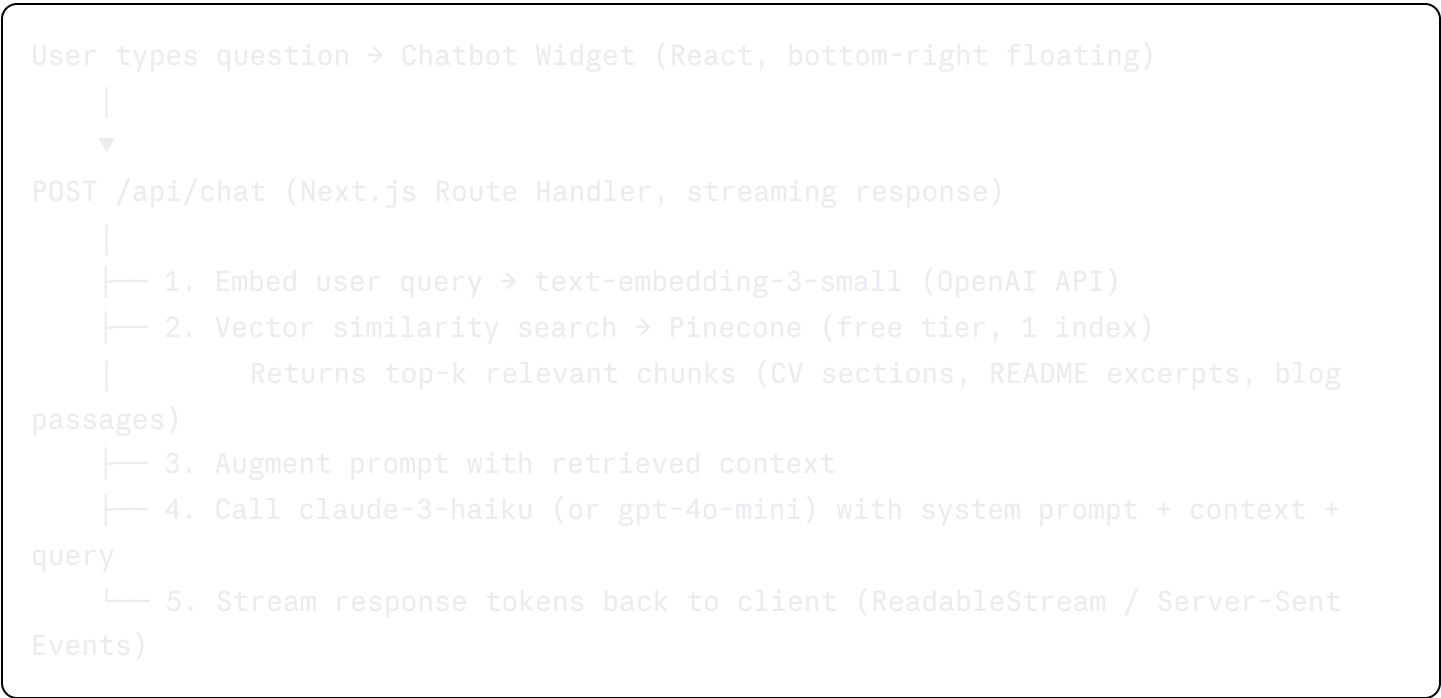
async function getGithubStats(repo: string) {
  try {
    const res = await fetch(`https://api.github.com/repos/${repo}`, {
      headers: { Authorization: `Bearer ${process.env.GITHUB_TOKEN}` },
      next: { revalidate: 3600 }
    });
    if (!res.ok) throw new Error('GitHub API failed');
    const data = await res.json();
    return { stars: data.stargazers_count, commits: await getCommitCount(repo) };
  } catch {
    // Return fallback — never crash the page for a stat widget
    return { stars: null, commits: null, error: true };
  }
}
```

19.3 Offline / Network Failure UX

- Contact form: saves draft to `localStorage` so user doesn't lose their message on network failure
 - Admin CMS: all writes use optimistic UI with rollback on Firestore error
 - Firestore offline persistence enabled for admin panel (Firestore SDK `enableIndexedDbPersistence`)
-

20. TRD — Phase 2: AI Chatbot Integration

20.1 Architecture



20.2 Knowledge Base Documents

Document	Source	Update Frequency
CV / Resume	Firebase Storage PDF	On CV upload (admin trigger)
Project READMEs	GitHub API (fetched and parsed)	Nightly scheduled job
Blog Posts	Firestore (all published)	On blog publish (admin trigger)

20.3 Admin Knowledge Base Management

An "AI Knowledge Base" section in the admin panel allows:

- Manual trigger to re-index all documents
- View indexed document list with chunk counts
- Add/remove specific documents from the index
- Clear and rebuild index

20.4 Chatbot UI Specifications

- Floating button (bottom-right, fixed position) with a chat bubble icon
 - Opens a 380x500px chat panel (full-screen on mobile)
 - Message history persisted in `sessionStorage` (cleared on tab close)
 - Streaming response shown with animated typing indicator
 - Suggested starter questions: "What AI projects have you built?", "Tell me about your RAG experience", "Are you open to remote roles?"
 - Rate limited: 10 messages per session to control API costs
-

21. Appendix A — Project File Structure

```
portfolio/
├── .github/
│   ├── workflows/
│   │   ├── ci.yml
│   │   └── deploy.yml
├── .husky/
│   └── pre-commit
├── .lighthouserc.json
├── app/
│   ├── layout.tsx           ← Root layout, fonts, theme script
│   ├── page.tsx             ← Homepage (SSG)
│   ├── not-found.tsx        ← Custom 404
│   ├── error.tsx            ← Global error boundary
│   ├── loading.tsx          ← Global loading skeleton
│   ├── sitemap.ts           ← Dynamic sitemap
│   ├── robots.ts            ← robots.txt
│   ├── blog/
│   │   ├── page.tsx         ← Blog feed (SSG + ISR)
│   │   └── [slug]/
│   │       ├── page.tsx     ← Blog post (SSG + ISR)
│   │       └── loading.tsx
│   └── admin/
```

			└─ layout.tsx	← Admin layout (sidebar, session check)
			└─ login/	
			└─ page.tsx	
			└─ dashboard/	
			└─ page.tsx	
			└─ projects/	
			└─ page.tsx	← Project list
			└─ new/page.tsx	← Create project
			└─ [id]/edit/page.tsx	← Edit project
			└─ blog/	
			└─ page.tsx	← Blog list
			└─ new/page.tsx	
			└─ [id]/edit/page.tsx	
			└─ comments/	
			└─ page.tsx	← Moderation queue
			└─ timeline/	
			└─ page.tsx	
			└─ inbox/	
			└─ page.tsx	← Contact submissions
			└─ cv/	
			└─ page.tsx	← CV upload
			└─ settings/	
			└─ page.tsx	
			└─ api/	
			└─ github-stats/	
			└─ route.ts	
			└─ contact/	
			└─ route.ts	
			└─ auth/	
			└─ session/route.ts	← Set/clear HttpOnly cookie
			└─ logout/route.ts	
			└─ chat/	
			└─ route.ts	← Phase 2: RAG streaming endpoint
			└─ components/	
			└─ public/	
			└─ Hero/	
			└─ Timeline/	
			└─ Projects/	
			└─ Tier1Card.tsx	
			└─ Tier2Card.tsx	
			└─ Tier3List.tsx	
			└─ Blog/	
			└─ BlogCard.tsx	
			└─ BlogPost.tsx	

```

|   |   |   └─ CommentSection.tsx
|   |   └─ ContactForm.tsx
|   |   └─ Navbar.tsx
|   |   └─ Footer.tsx
|   └─ admin/
|       └─ AdminSidebar.tsx
|       └─ ProjectForm.tsx
|       └─ BlogEditor.tsx
|       └─ CommentModerationTable.tsx
|       └─ FileUploadInput.tsx
|   └─ ui/                                ← Shared primitives (Button, Input, Modal,
Toast)
└─ lib/
    └─ firebase/
        └─ client.ts                    ← Firebase client SDK init
        └─ admin.ts                    ← Firebase Admin SDK init (server-only)
        └─ converters.ts                ← Firestore type converters
    └─ auth/
        └─ verify-session.ts
    └─ actions/                        ← All Server Actions
        └─ projects.ts
        └─ blogs.ts
        └─ comments.ts
        └─ contact.ts
        └─ auth.ts
    └─ github.ts                      ← GitHub API utilities
    └─ email.ts                      ← EmailJS/Resend utilities
    └─ utils.ts
└─ types/
    └─ index.ts                      ← All TypeScript interfaces
└─ public/
    └─ og-image.png
    └─ favicon.ico
    └─ icons/
└─ middleware.ts
└─ next.config.ts
└─ tailwind.config.ts
└─ tsconfig.json
└─ firestore.rules
└─ storage.rules
└─ .env.local                        ← (gitignored)
└─ .env.example                    ← (committed - sanitized)
└─ package.json

```

22. Appendix B — Environment Variables

bash

```
# .env.example - All required environment variables
# Copy to .env.local and fill in your values
# NEVER commit .env.local to git

# — Firebase Client SDK (public - prefixed with NEXT_PUBLIC_) —
NEXT_PUBLIC_FIREBASE_API_KEY=
NEXT_PUBLIC_FIREBASE_AUTH_DOMAIN=
NEXT_PUBLIC_FIREBASE_PROJECT_ID=
NEXT_PUBLIC_FIREBASE_STORAGE_BUCKET=
NEXT_PUBLIC_FIREBASE_MESSAGING_SENDER_ID=
NEXT_PUBLIC_FIREBASE_APP_ID=

# — Firebase Admin SDK (server-only - NO NEXT_PUBLIC_ prefix) —
FIREBASE_ADMIN_PROJECT_ID=
FIREBASE_ADMIN_CLIENT_EMAIL=
FIREBASE_ADMIN_PRIVATE_KEY= # Wrap in quotes, preserve \n newlines

# — Admin Security —
ADMIN_UID= # Firebase Auth UID of the sole admin account

# — GitHub API (server-only) —
GITHUB_TOKEN= # Fine-grained PAT - read:public_repo only

# — Email —
EMAILJS_SERVICE_ID=
EMAILJS_TEMPLATE_ID=
EMAILJS_PUBLIC_KEY=
# OR if using Resend:
RESEND_API_KEY=
CONTACT_EMAIL= # Your inbox email

# — Phase 2: AI Chatbot (add when implementing Phase 2) —
OPENAI_API_KEY=
PINECONE_API_KEY=
PINECONE_INDEX_NAME=

# — Vercel (set in Vercel dashboard, not in .env.local) —
# VERCEL_URL - auto-injected by Vercel
# VERCEL_ENV - auto-injected by Vercel (development/preview/production)
```

Version History

Version	Date	Changes
1.0	Initial	Base PRD/TRD from existing document
2.0	Current	Exhaustive expansion: full schema, security rules, auth architecture, CI/CD, SEO, file structure, environment config, Phase 2 AI chatbot